

[Sistema Inteligente de Control de Temperatura con Regulación PWM]

Docente: Diego Miranda Henríquez.

Integrantes: Mauricio Henríquez, Benjamin Lazo,

Gabriel Espinoza, Diego Allendes.

Asignatura: ISF-215 Hardware digital - Tarea 1.

Introducción.

- El control térmico es una necesidad fundamental en entornos industriales y de computación para garantizar la integridad física de los equipos. El informe presentado detalla el diseño e implementación de un prototipo de sistemas embebidos basado en la plataforma de Arduino UNO, cuyo objetivo es regular de manera autónoma la velocidad de un ventilador en función de la temperatura ambiental, indicando visualmente su estado de operación (indicando estado activo / pasivo).
- Siguiendo el enfoque, se partió la necesidad de la capa de aplicación para seleccionar sensores (*TMP36*) y actuadores (*Motor DC* y *LEDs*) adecuados. Posteriormente, el diseño se ajustó a las capacidades de lectura analógica, procesamiento de señales digitales (*debouncing*) y generación de modulación por ancho de pulsos (*PWM*) del microondas *ATmega328P* subyacente. El sistema final no solo cumple con los requerimientos funcionales del cliente interno, sino que permite un modo de cancelación de alta prioridad.

Diseño de Sistema.

-El sistema se compone de tres bloques principales, orquestados por el microcontrolador.

1. Bloque de Entrada (Control y Contabilidad):

- a. **Sensor de Temperatura (*TMP36*):** Actúa como transductor, entregando un voltaje linealmente proporcional a la temperatura.
- b. **Pulsador Tactil:** Entrada digital que permite al usuario alternar entre el modo de control automático y el modo actual.

2. Bloque de procesamiento (*Arduino UNO/ATMega328P*):

Convierte señales analógicas a digitales mediante su *ADC* interno.

- a. Filtra lecturas ruidosas del pulsador mediante técnicas de *Debouncing* para software.
- b. Ejecuta el algoritmo de control lógico para determinar el ciclo de trabajo (*duty Cycle*) del *PWM*.

3. Bloques de Salida (*Actuadores e interfaz HMI*):

- a. **Driver de motor L298N + Motor DC:** El driver amplifica la señal *PWM* de control del arduino para proporcionar la corriente necesaria que regula la velocidad del ventilador.
- b. **Controladores *LED* (verde y rojo):** Interfaz visual que comunica el estado del sistema. Verde para el estado “pasivo” (Ventilador apagado) y rojo para el estado “activo” (ventilador en movimiento).

Esquema de Conexiones.

Aunque en esta versión de texto se omite la representación gráfica, el diagrama físico (*Tinkercad*) establece las siguientes conexiones fundamentales:

- **Señales Analógicas:** El pin *Vout* el sensor *TMP36* se conecta al pin *A0* del Arduino.
- **Señales Digitales de Entrada:** El pulsador se conecta al pin 2. Se aprovecha la resistencia de *pull-up* interna del microcontrolador, conectando el otro lado del pulsador a *GND*.
- **Señales Digitales de Salida:** El ánodo del *LED* verde se conecta al pin 7 y el *LED* rojo se conecta al pin 6 (Ambos protegidos por resistencias de 220 OHM).
- **Control *PWM* y Potencia:** El pin 9 (capaz de generar *PWM* por hardware en el *ATmega328P*) se conecta al pin *ENA* del puente *H L298N* para controlar la velocidad. Los pines 8 y 4 del arduino se conectan a *IN1* e *IN2* del driver para establecer el sentido del giro.

Descripción del Código.

El código base, desarrollado en *C++*, se estructura mediante técnicas de *Polling* continuo. El flujo principal dentro de la función `loop()` evalúa constantemente el estado del botón y la lectura del sensor analógico.

- **Manejo de la lectura analógica y conversión:** El *ADC* convierte el voltaje del sensor *TMP36* en un número entero. para relacionar este valor del *ADC* (0-1023) con una magnitud física real (°C), el código implementa la siguiente normalización:

```
int adc = analogRead(pinSensor);  
float voltaje = adc * 5.0 / 1023.0;  
float temperatura = (voltaje - 0.5) * 100.0;
```

- **Lógica de Control (*PWM*):** La función `calcularPWM(float temperatura)` definiendo el comportamiento del actuador. Existe una condición de umbral (`umbral = 30.0`). Si la temperatura es menor, el ventilador se detiene (Retorna `0`). Si se encuentra entre 30°C y 40°C, se realiza un escalamiento proporcional usando la función `map()` nativa de arduino:

```
if (temperatura < 40) {  
    return map(temperatura, 30, 40, 90, 180);  
}
```

Si la temperatura supera los 40°C, el ventilador opera a máxima potencia (`255`). Para asegurar una lectura limpia de la entrada digital (pulsador) sin recurrir a hardware adicional como capacitores, se implementó una función `manejarboton()` que evalúa si el estado de la señal se ha mantenido estable por un `tiempodebounce = 50` milisegundos utilizando la función de interrupción de tiempo `millis()`.

Análisis de Arquitectura ATmega328P.

- **Uso de flash (Memoria de programa):** Se estima un consumo de 2.5 a 3.5 KB de los 32KB totales disponibles (~10%). Al depender sólo de operaciones aritméticas y de comunicación serial, queda un margen muy amplio para algoritmos futuros.
- **Uso de SRAM (Variables globales):** El código aloja 12 variables globales simples, ocupando apenas unos 27 bytes de los 2KB disponibles(~1.3%). El sistema opera sin riesgo de *stack overflow*.
- **Frecuencia de muestreo y precisión del ADC:** El ADC de 10 bits (5V/1024) ofrece una resolución aproximada de 4.88mV, lo que otorga una precisión teórica de 0.488°C junto al sensor TMP36. Si bien el microcontrolador puede muestrear a 9600 Hz, una tasa suficiente para variables físicas lentas como la temperatura.
- **Limitaciones para expansión:** El principal cuello de botella de la función de bloqueo `delay(300)`. Este retardo detiene el procesamiento, pudiendo causar pérdida de eventos en la entrada del pulsador (*debouncing*). Para escalar el proyecto, se debe refactorizar el código hacia una lógica no bloqueante con `millis()` o usar interrupciones por hardware (*INT0*).

Conclusiones.

El desarrollo del prototipo demostró empíricamente cómo interactúan las capas lógicas y físicas de un microcontrolador. Un aprendizaje clave fue comprender la necesidad de estabilizar las señales físicas del mundo real; el rebote mecánico del interruptor táctil habría generado inestabilidad en la máquina de estados del modo manual sin el algoritmo de *debouncing* implementado en `manejarBoton()`.

Asimismo, el desafío de parametrizar adecuadamente el PWM evidenció que el control analógico exige una sincronización matemática precisa (`map`) para que las transiciones de los actuadores sean escalonadas y no binarias. Como resultado, el sistema cumple íntegramente con los requisitos operacionales propuestos en la rúbrica del proyecto, logrando un diseño robusto y justificado en base a la arquitectura disponible.